

This is an interesting paper tackling a real and important problem in HPC scheduling. The core idea — using MCTS as a training-free, goal-configurable alternative to DRL schedulers — is well-motivated, and the system-draining angle is a genuine contribution. Below are my constructive comments organized by importance.

Major Concerns

1. The evaluation lacks statistical rigor and comparison breadth.

The paper compares MARS against FCFS, SJF, WFP, F1, RLScheduler, and Random, but several things are missing:

- No confidence intervals, error bars, or significance tests anywhere. Given that MCTS is stochastic (random rollouts, parallel trees), you should report results across multiple seeds. Right now Figure 9's box plots show distribution across jobs, not across runs of the scheduler.
- No comparison to other MCTS-for-scheduling baselines (e.g., the job-shop MCTS work you cite as [24], [25]). Even an ablated version would help readers understand which design choices matter.
- The Random baseline is interesting but underexplained — does it pick uniformly across the 161 actions, or weight by something? If MARS-CW only modestly beats Random in Figure 9, that's a significant finding that deserves discussion rather than a quick mention.

2. The RLScheduler comparison feels weak and may be unfair.

You note RLScheduler "does not reach full convergence" and acknowledge this aligns with the original paper. But this then becomes a strawman comparison. Consider:

- Training RLScheduler on Polaris-2023 and testing on Polaris-2024 would be a fairer same-system test.
- The bounded slowdown objective in RLScheduler differs from your CW reward — you're partly comparing different objectives, not just different methods.
- If the central claim is "MARS doesn't need training and DRL does," then showing the *cost* of DRL training (compute hours, data requirements) versus MARS's 256-core $\times$ 15s overhead would strengthen the argument quantitatively.

3. Computational cost is under-reported.

MARS uses 256 CPU cores per scheduling decision. This is a substantial resource commitment that production sites would want quantified:

- What fraction of system capacity does this represent on Polaris (560 nodes) vs. Theta (4360 nodes)?
- How does performance degrade with fewer cores (e.g., 32, 64, 128)? A scaling study would be very valuable.
- What happens if $T < 15s$? A sensitivity analysis on T would address concerns about the time budget assumption.
- Energy cost comparison versus heuristics would also be relevant given HPC's energy concerns.

4. The MDP formulation has a subtle issue worth clarifying.

You define the action as a reordering of the window, but the reward depends on what runs *during the transition* to the next scheduling cycle. This means rewards depend on job arrivals, completions, and other stochastic events that happen between cycles. Is the simulator deterministic given the action, or does it sample? If deterministic (replay-based), this should be stated; if not, the rollout variance needs discussion. Related: how do you handle the partial observability of true runtime (you mention using walltimes for planning but actual runtime for replay — this asymmetry deserves more analysis since walltime overestimation is a known issue in HPC).

Methodology Issues

5. The branching-factor claim is somewhat misleading.

Equation (6) states " $16 \times 10 + 1 = 161$ " but Figure 16 shows the *effective* branching factor is much smaller after node merging — which is good, but the paper sometimes frames 161 as the operative number. Be consistent: use the effective number when discussing search complexity.

6. The "depth $D = 50$ " choice is unjustified.

Why 50? A sensitivity analysis ($D = 10, 25, 50, 100$) showing diminishing returns would justify the choice. The same applies to $\gamma = 0.99$ and the parallel-tree count of 250.

7. The reward functions are defined but not deeply analyzed.

Equation (7), $R_{CW} = 1/(1+W_t)$, is bounded in $[0,1]$ which is good for UCT, but it has a non-linear relationship to wait time that could create odd incentives (e.g., reducing wait from 100h to 50h gives almost no reward improvement). Discuss the implications. Why not use $-W_t$ with normalization, or a log-scaled reward?

Presentation and Writing

8. Figures need more polish.

- Figure 3 caption doesn't explain what "Backfill Fraction" means — fraction of jobs? fraction of node-hours?
- Figures 4 and 5 are conceptually similar but use different visual languages; consider unifying.
- Figure 6 is dense and hard to parse — consider splitting into separate panels or using a clearer color scheme.
- Figures 14 and 15 are nearly identical structurally; consider combining or making the comparison explicit (e.g., side-by-side per system).
- Figure 11 caption could explicitly state the takeaway: "MARS-CU achieves the highest median utilization on Theta."

9. Some claims need tempering or more support.

- "MARS converges within seconds" (Conclusion) — MCTS doesn't really *converge* in the DRL sense. It does a bounded search. Reword.
- "More consistent metric-specific optimization than static heuristics which inevitably trade one metric for another" — MARS *also* trades metrics (CW vs. CU configurations explicitly do this). The argument should be that MARS makes the trade *explicit and configurable*, not that it avoids the trade.
- The claim that DRL has "poor cross-system stability" is supported only by one citation and your own RLScheduler results. This is a strong claim for the field; either soften it or expand the evidence.

10. Related Work is thin.

Section VII would benefit from:

- More discussion of MuZero/AlphaZero-style approaches that combine MCTS with learned value functions — could MARS benefit from a learned rollout policy?
- Recent work on LLM-based or hybrid scheduling.
- Work on online/anytime planning under time budgets (which is exactly what MARS does).
- Engelmann/Klusáček-style work on production-scheduler simulation that goes beyond CQSim.

11. Writing-level issues.

- "looses out" → "loses out" (Section VI-A and elsewhere).
- "back-filling" / "backfilling" / "back filling" — pick one.
- Inconsistent capitalization of "Backpropagation" vs. "backpropagation."
- "sleaf" appears once where "s_leaf" was intended (Equation 3 area).
- "scheduled multiple M or S jobs" — verb tense issue.
- Several long sentences in the introduction would benefit from being split.
- Table I labels "Heur." and "Opt." are not defined until Section VII.

Suggestions for Strengthening the Contribution

12. Lead with the most novel finding.

The paper's most striking and novel result is that MARS *learns to drain the system proactively* — this is qualitatively different from what heuristics or DRL do. Right now this finding is buried in Section VI-C. Consider:

- Adding a teaser plot in the introduction showing a scheduling timeline with drain cycles annotated.
- Restructuring to lead Section VI with the drain analysis.
- Adding a case study: pick one specific maintenance window and walk through MARS-CU's decisions hour-by-hour versus WFP's decisions, showing the divergent strategies.

13. Address the obvious next question: hybrid rewards.

A reviewer will ask: can MARS optimize a weighted combination of CW and CU? Even a brief experiment with $R = \alpha \cdot R_{CW} + (1-\alpha) \cdot R_{CU}$ sweeping α would meaningfully extend the paper and showcase the configurability claim.

14. Discuss failure modes.

When does MARS perform worse than heuristics? The Polaris 48-hour utilization plot (Figure 11) shows high variance — when does MARS-CU actually fail there? An honest section on limitations would strengthen credibility.

15. Production deployment story is missing.

You claim scheduler-agnostic design but don't show integration with PBS or Slurm. Even a short section describing the API surface (how priorities flow from MARS to the scheduler, how state is queried) would address the practical-deployment concern. Has this been tested in any live or shadow-mode deployment, even briefly?

Minor

- Section III-B Equation (1): the LaTeX is slightly malformed in the rendered version.
- The " $\gamma = 0.99$ " with depth 50 means rewards at the leaf are weighted by $0.99^{50} \approx 0.6$ — meaningful but not negligible. Worth a brief note.
- Define ALCC, INCITE, DD on first use.
- Table II: clarify whether all 17 heuristics were run as standalone baselines or only the listed ones. If only some were used as baselines, why those specifically?
- Section IV-A: the claim that heuristic behavior "transfers reliably across systems" deserves a figure or table reference.

Summary

The core contribution is real and the system-draining result is genuinely novel for HPC scheduling literature. The main weaknesses are (a) evaluation rigor — single runs, no significance tests, weak DRL comparison; (b) under-reported computational cost; and (c) some over-reaching claims that could be tightened. Addressing the evaluation concerns and leading with the drain-behavior finding would substantially strengthen the paper. The work seems publishable at a strong HPC venue with these revisions.